

JavaScript

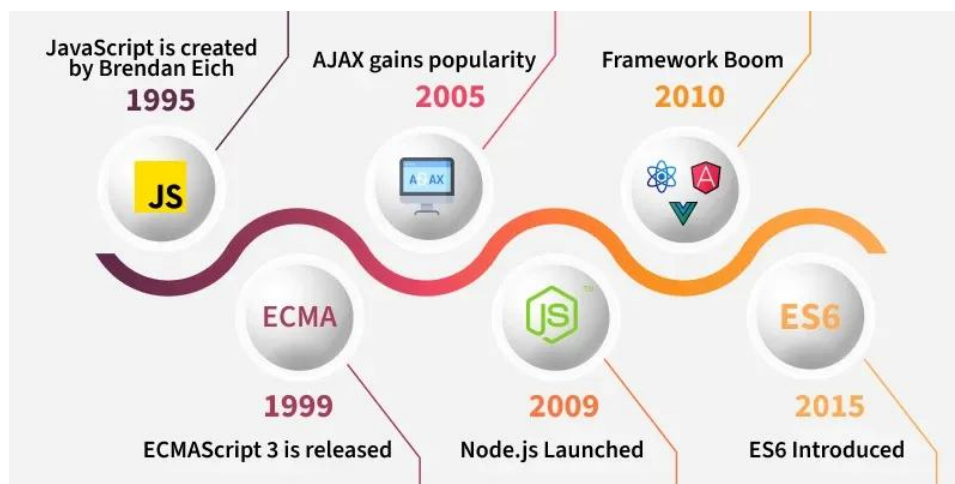
Introduction to JavaScript :-



- JavaScript (js) is a light-weight object-oriented programming language
- JavaScript is not a compiled language
- enables dynamic interactivity on websites when applied to an HTML document.

History & Names -

- Developed by Brendan Eich in 10 days (1995).
- First name: Mocha
- Second name: LiveScript
- Third name: JavaScript
- Standardized name: ECMAScript (ES)



Platforms –

JavaScript works on three levels:

1. Frontend - Browser scripting (with HTML, CSS).
2. Backend - Using Node.js (server-side).
3. Middleware (Mid-end) - Between frontend and backend (e.g., APIs, Express.js).

Application of JavaScript :-



- Client-side validation
- Dynamic drop-down menus
- Displaying date and time
- Displaying pop-up windows and dialog boxes (like an alert dialog box, confirm dialog box and prompt dialog box)
- Displaying clocks etc.

How to Write Program in JavaScript :-



Demo.html

```
<script>
  document.write("Hello JavaScript");
  alert("Hello JavaScript ");
</script>
```

Output Methods in JavaScript :-

1. console.log() - Shows output in the browser's console (for debugging).

Example - console.log("Message in console");

2. alert() - Displays output in a popup alert box.

Example - alert("This is an alert message!");

3. document.write() - Writes output directly into the HTML page.

Example - document.write("Hello, World!");

Input Methods in JavaScript :-

1. `prompt()` - Takes input from the user using a popup dialog box.

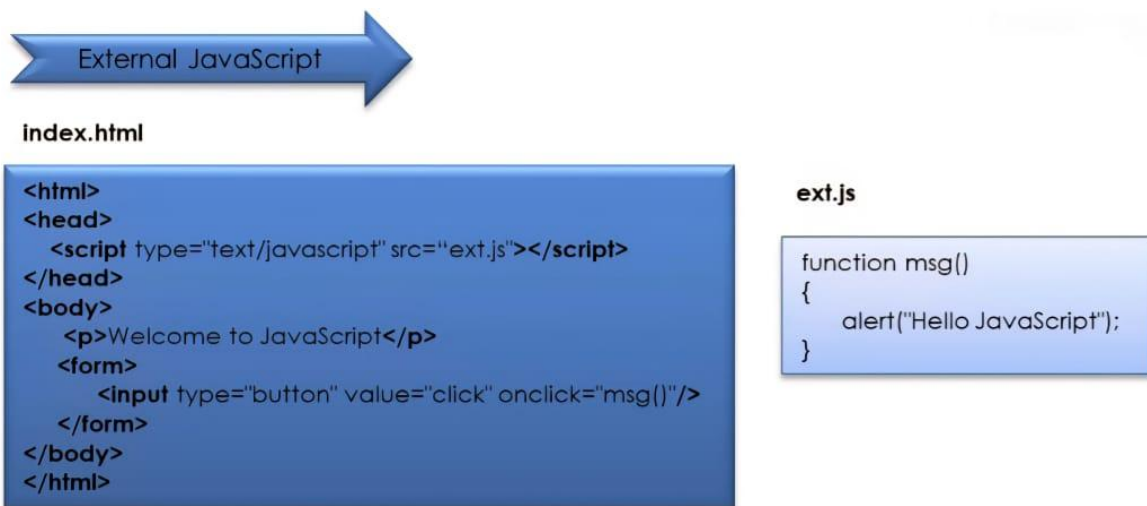
```
var name = prompt("Enter your name:");
```

```
alert("Hello, " + name);
```

Ways to Insert JavaScript :-

There are the two main ways to insert JavaScript in a webpage.

External JavaScript :-



Internal JavaScript :-

- Internal JavaScript means writing JavaScript code inside an HTML file.
- The code is placed within the `<script> ... </script>` tags.
- Usually, it is written inside the `<head>` or `<body>` section of the HTML page.
- It is directly linked with the webpage without using an external file.

Structure -

```
<script>

// JavaScript code here

</script>
```

Synchronous JavaScript :-

- Execution happens line by line (one at a time).
- Each statement waits for the previous one to complete before running.
- Blocking in nature - If one task takes time, it stops the rest until finished.

Example -

```
console.log("Task 1");  
console.log("Task 2");
```

Output -

Task 1

Task 2

Asynchronous JavaScript

- Allows multiple tasks to run without waiting for others to finish.
- Non-blocking → long tasks (like API calls, timers, file reading) don't stop execution.
- Uses callbacks, promises, async/await to handle tasks.

Example with setTimeout -

```
console.log("Task 1");  
setTimeout(() =>  
{  
  console.log("Task 2 (after 2 seconds)");  
}, 2000);  
console.log("Task 3");
```

Output:

Task 1

Task 3

Task 2 (after 2 seconds)

Variable in JavaScript :-

- A **variable** is simply a name of storage location.



- Name must start with a letter (a to z or A to Z), underscore (_), or dollar (\$) sign.
- After first letter we can use digits (0 to 9)
- JavaScript variables are case sensitive

```
<script>
var x = 10;
var y = 20;

var z=x+y;

document.write(z);
</script>
```

Types of Variable :-



1. Local Variable –



- Declared inside block or function
- Accessible within the function or block only

```
<script>

function display()
{
    var a=10; //local variable
}

</script>
```

2. Global Variable –



- Declared outside the function
- A **global variable** is accessible from any function

```
<script>

var a=10; //Global variable

function display()
{
    document.writeln(a);
}

</script>
```

Keywords in JavaScript :-

- Keywords are the reserved words in JavaScript.
- They have special meaning and purpose in the language.
- Keywords cannot be used as variable names, function names, or identifiers.
- Examples: var, let, const, if, else, for, while, switch, return, etc.
- They are used to define structure and control flow in programs.

abstract	delete	function	null	throw
boolean	do	goto	package	throws
break	double	if	private	transient
byte	else	implements	protected	true
case	enum	import	public	try
catch	export	in	return	typeof
char	extends	instanceof	short	var
class	false	int	static	void
const	final	interface	super	volatile
continue	finally	long	switch	while
debugger	float	native	synchronized	with
default	for	new	this	

Difference Between var, let & const in Js :-

VAR, LET, AND CONST

STATEMENT	BLOCK SCOPED	TDZ	CREATES GLOBAL PROPERTY	REASSIGNABLE	REDECLARABLE
var	✗	✗	✓	✓	✓
let	✓	✓	✗	✓	✗
const	✓	✓	✗	✗	✗